

Software Engineering Foundations

1. Software Product & Software Engineering

1.1 What is Software?

- A **product** that software professionals design, develop, deliver, and support over time.
- A **tool or environment** to support the development of other software products.

Software consists of:

- Programs (code instructions)
- Execution-time data (inputs, outputs, UI content, messages)
- Configuration files
- Documentation

So:

Software = Programs + Execution Data + Configurations + Documentation

1.2 Dual Role of Software

Software acts as:

1. **Software as a Product**

Used directly by users (banking system, ERP, apps)

2. **Software as a Development Environment / Tool**

Used to build other software (compilers, IDEs, testing tools)

1.3 Characteristics of Software vs Hardware

Attribute	Hardware	Software
Nature	Manufactured	Engineered/ Developed
Physical wear	Yes	No
Failure reason	Physical degradation	Complexity & changes
Production style	Mass-production	Often custom-built

Important Principle

Software does not wear out — it deteriorates due to frequent changes and complexity.

1.4 Key Characteristics

1. **Software is engineered — not manufactured**

- No assembly line production
- Each project is unique

2. **Software does not “wear out”**

- Hardware fails due to physical stress

- Software never wears out physically

3. **Software deteriorates over time**

- Because of:
 - Frequent updates
 - Bug fixes
 - New features
- This leads to **complexity and instability**

4. **Most software is custom-built**

- Unlike hardware mass-production
- Though reuse is increasing

1.5. **Categories of Software**

Software can be grouped as follows:

1. **System Software**

- Supports operation of a computer system.
- Examples:
 - Operating systems
 - Compilers
 - File managers



2. **Application Software**

- Developed for business or user needs.
- Example:
 - Banking software
 - Billing systems
 - MS Word

3. **Engineering & Scientific Software**

- Used for:
 - Research
 - Engineering calculations
 - Simulations

4. **Embedded Software**

- Built into hardware devices.
- Examples:

- Washing machine controller
- Car braking system
- Routers

5. Product-Line Software

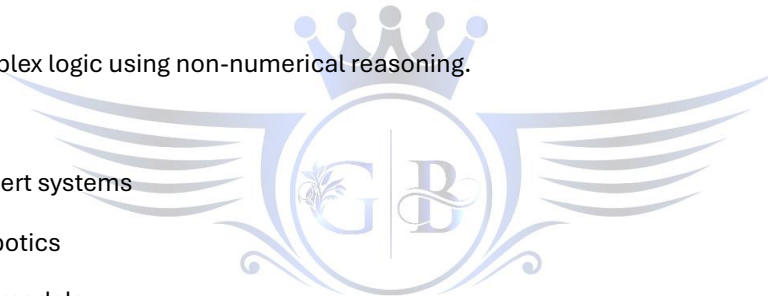
- Developed for specific market needs.
- Example:
 - Inventory management software

6. Web Applications

- Executed over the internet.
- Example:
 - E-commerce systems
 - Portals
 - Online banking

7. AI Software

- Solves complex logic using non-numerical reasoning.
- Example:
 - Expert systems
 - Robotics
 - ML models



2. Software Engineering

2.1 Definition

Software Engineering is the **systematic, disciplined, and measurable** application of engineering methods to software development, operation, and maintenance.

Key Points

- It brings **engineering discipline** to software development.
- Ensures software is:
 - Reliable
 - Maintainable
 - Cost-effective
 - Delivered on time

2.2 Why Software Engineering Is Needed

Because software projects face challenges like:

- Delays
- Cost overruns
- Poor quality
- Maintenance difficulty
- Failure to meet user needs



This historic issue was called the:

Software Crisis (1968 – NATO Conference)

Causes:

- Increasing system complexity
- Lack of proper methodologies
- Poor project control
- Weak testing
- Unrealistic schedules

Software Engineering emerged to solve this.

2.3 History Timeline (Simplified)

Period	Key Events
1941	First digital computer
1950s	Programming languages introduced
1960s	Unix OS — modularity begins
1968	Term "Software Engineering" introduced
1970s	Structured programming & OOP introduced
1980s	SDLC formalization
1990s	Internet & Open Source rise
2000s	Agile methodologies
Today	Cloud, DevOps, AI-driven systems



3. Types of Software Projects

Software projects are broadly categorized into **Development, Maintenance, and Conversion projects.**

A. Development Projects

These involve **building new software applications.**

1. Full Development

This includes the **entire lifecycle from beginning to end**, such as:

- Requirement Analysis
- Design
- Coding
- Testing
- Deployment
- Post-implementation support and warranty period

Example:

Building a new banking application from scratch.

2. Part Development

Only **partial development of the system is done**, not the entire lifecycle.

Examples

- Developing only UI layer
 - Developing only backend module
 - Outsourced module development
-

B. Maintenance Projects

These involve **working on existing software** after deployment.

Maintenance includes three major types:

1. Regular Maintenance

Modifying software to:

- Correct issues (bug fixing)
- Adapt to environmental changes
- Extend existing functionality

This is mostly **reactive in nature.**

Examples

- Fixing production defects
 - Updating software to work with updated OS
 - Modifying logic based on new compliance rules
-

2. Enhancements

Enhancement involves **adding significant new functionality or performance improvements**.

This is **proactive and value-adding**, unlike maintenance which is reactive.

Examples

- Adding new modules
 - Integrating payment gateway
 - Improving user interface
 - Performance tuning
-

3. Production Support

Objective: **Keep system running smoothly in real-time.**

Activities include:

- Monitoring uptime
- Handling incidents
- Resolving user issues
- Maintaining SLAs
- Performing quick fixes

This is **continuous support work**.

C. Conversion Projects

These involve **changing technology, platform, or structure** of existing systems.

1. Re-Engineering

Rebuilding an existing system **partially or completely** to:

- Improve performance
- Make system scalable
- Improve structure

Core business functionality **remains same**.

Examples

- Refactoring poorly structured code
 - Rewriting COBOL system into Java
 - Optimizing code performance
-

2. Migration

Moving an application or data to a **different environment**, usually with **minimal code change**.

Examples

- Moving from on-premise to cloud
 - Migrating database from Oracle to SQL Server
-

3. Porting

Making software available on **multiple platforms**.

Examples

- Windows app ported to Linux
 - iOS app ported to Android
-



4. Software Process & Process Models

4.1 What is a Software Process?

A **software process** defines:

Who does what, when, and how during software development.

It ensures repeatable, controlled execution.

4.2 Generic Software Process Framework

Every project contains five core activities:

1. **Communication** — requirements, stakeholder interaction
 2. **Planning** — effort, schedule, risk, resources
 3. **Modeling / Design** — architecture & structure
 4. **Construction** — coding + testing
 5. **Deployment** — delivery + feedback + support
-

4.3 Umbrella Activities

These activities support all phases:

- Project Tracking and Control
 - Risk Management
 - Quality Assurance
 - Technical Reviews
 - Measurement and Metrics
 - Configuration Management
 - Reusability Management
 - Documentation
-



5. Software Development Life Cycle (SDLC)

Definition (IEEE)

“SDLC is the period beginning from the decision to develop software until the software is delivered and implemented.”

IN simple : SDLC defines the complete software lifecycle — from **requirement to maintenance..**

This includes:

- Requirement phase
- Design
- Development
- Testing
- Deployment
- Warranty / Maintenance

There can also be separate lifecycles such as:

- Testing lifecycle
- Deployment lifecycle
- Maintenance lifecycle
- Customization lifecycle

Common SDLC Models (Process Models)

- Waterfall
- Iterative
- Spiral
- V-Model
- Incremental
- Agile
 - XP
 - Scrum
 - Kanban / Lean
 - Crystal
 - DSDM
 - FDD
 - RUP

Each model defines the **order & feedback level of development activities**.

Types of Process Models

A. Prescriptive Process Models

They:

- Define structured phases
- Provide full control and documentation
- Are stable and predictable

Includes:

- Waterfall
 - Incremental
 - RAD
-

B. Evolutionary Models

They allow:

- Iteration
- Feedback loops
- Continuous refinement

Includes:

- Prototyping
 - Spiral
 - Many Agile approaches
-



Models

1. Waterfall Model (Linear Sequential Model)

A **sequential phase-based model** where each phase completes before the next starts.

Phases

1. Requirement Analysis
2. Design
3. Coding
4. Testing
5. Deployment
6. Maintenance

Strengths

- Simple and structured
- Clear milestones
- Suitable for stable requirements
- Good documentation

Weaknesses

- No early working model
- Late customer feedback
- Very rigid
- High risk if requirement changes

Best suited when

- Requirements are clear and stable
- Technology is known
- Quality is critical

8. Incremental Model

Software is delivered **in small usable increments**.

Each increment follows:

- Requirement
- Design
- Coding
- Testing
- Deployment

Key Features

- Partial software delivered early
- Customer feedback after each increment
- First release = core product

Strengths

- Early delivery
- Reduced risk
- Better user feedback
- Customers see progress
- Flexible to requirement changes

Weaknesses

- Requires good planning
- Integration challenges
- Scope creep risk

Best suited when

- Requirements evolve
 - Time-to-market is important
-

2. Rapid Application Development (RAD)

A **fast development model** using:

- Prototyping
- User workshops
- CASE tools

Phases

- Requirements planning
- User design
- Construction
- Cut-over

Strengths

- Very fast delivery
- High user involvement
- Reduced risk
- Early validation

Weaknesses

- Requires skilled team
- Not suitable for large/critical systems
- Requires user availability

Best suited when

- Requirements are moderately clear
 - Time is critical
-

3. Prototyping Model

A **temporary prototype** is built to:

- Clarify requirements
- Demonstrate system behavior

Then final system is built.

Strengths

- Excellent user feedback
- Reduces requirement ambiguity
- Encourages learning

Weaknesses

- Risk of “quick & dirty coding”
 - Poor documentation
 - Customers may want prototype as final product
-

4. Spiral Model

Combines:

- Waterfall structure
- Prototyping
- Risk analysis

Each loop includes:

- Objective setting
- Risk analysis
- Development
- Planning next phase



Strengths

- Best for high-risk projects
- Risk visibility
- Early user involvement

Weaknesses

- Expensive
 - Complex
 - Requires risk expertise
-

5. V-Model

Enhances Waterfall with **parallel testing planning**.

Every development phase has a **matching test phase**.

Example:

- Requirement ⇔ Acceptance Test

- Design ↔ System Test
- Module Design ↔ Unit Test

Strengths

- Strong testing discipline
- Good for critical systems

Weaknesses

- Rigid
 - Poor for changing requirements
-

6. Agile Development

Agile is **iterative and incremental development** focused on:

- Collaboration
- Customer feedback
- Adaptive planning

Examples:

- Scrum
- XP
- Kanban
- FDD
- DSDM
- Crystal
- RUP



Key Features

- Short iterations
 - Working software delivered frequently
 - Welcomes requirement changes
 - Continuous feedback
-

6. Requirements Engineering (RE)

6.1 What is a Requirement?

A **requirement** is:

- A necessary attribute of a system.
- A **statement that identifies a capability, characteristic, or quality factor** needed by a system so that it has value to the customer.
- It may range from:
 - High-level abstract business statements
 - To detailed mathematical / functional specifications.

In simple words:

A requirement describes **what the system must do or constraints under which it must operate.**

Brooks' Law Observation

"The hardest single part of building a software system is deciding what to build... If done wrong, the resulting system is crippled, and it is very difficult to fix later."

This is why Requirement Engineering is critical.

6.2. What is Requirement Engineering?

Requirements Engineering is the **systematic process of identifying, analyzing, documenting, validating, and managing software requirements.**

A requirement describes:

What the system must do OR constraints under which it must operate.

6.3. Characteristics of a Good Requirement

A requirement must be:

- **Necessary / Justifiable**
- **Verifiable** (can be tested)
- **Attainable / Feasible**
- **Unambiguous** (only one interpretation)
- **Complete**
- **Consistent** (no internal conflict)
- **Traceable**
- **Concise & clear**
- **Implementation-free**

- Requirement describes **WHAT**
- Design describes **HOW**

Requirement Engineering starts with what the user really needs — not what developers assume

6.4 Types of Requirements

1. Functional Requirements

These define:

- ✓ Services the system must provide
- ✓ How the system should react to inputs
- ✓ Behavior under specific situations

Examples:

- “System shall generate monthly reports.”
- “System shall validate login credentials.”

Functional requirements can be:

- User-level (high-level)
- System-level (detailed)

Examples:

- Validate login
- Generate reports



2. Non-Functional Requirements (NFRs)

Define **quality attributes & constraints**:

- Performance
- Security
- Reliability
- Availability
- Legal compliance
- Standards

Failure in NFRs often leads to system rejection — even if functionality works.

3. Domain Requirements

Domain-specific needs originating from the business environment.

Problems with domain requirements:

- **Understandability**
 - Written in business language developers may not understand

- **Implicitness**

- Domain experts assume some knowledge is “obvious” and do not mention it

6.5 Levels of Requirements

Level	Purpose	Document
Business Requirements	Business goals, CTQs	Vision / Charter
User Requirements	What users need	BRD / SRS
System Requirements	Technical details & constraints	SRS

6.6 Steps in Requirement Engineering

1. **Inception** — identify stakeholders, define scope & goals
2. **Elicitation** — gather requirements via interviews, workshops, observation, documents, prototyping
3. **Elaboration** — analyze, model workflows, rules, scenarios, data
4. **Negotiation** — resolve conflicts & prioritize
5. **Specification** — produce BRD / SRS baseline
6. **Validation** — verify completeness, correctness, testability
7. **Requirement Management** — control changes, maintain traceability

A **Requirement Traceability Matrix (RTM)** maps each requirement to corresponding design & test cases.

6.7 Requirement Engineering — Phases

1. **Inception** — Identify stakeholders, understand the problem context, and define the project scope at a high level.
 2. **Elicitation** — Gather needs, expectations, and constraints directly from stakeholders using structured techniques.
 3. **Elaboration** — Analyze, refine, and model the gathered requirements to make them structured and clear.
 4. **Negotiation** — Resolve conflicts, prioritize needs, and agree on what will actually be delivered.
 5. **Specification** — Document the finalized and approved requirements formally in an SRS/BRD.
 6. **Validation** — Verify that the documented requirements are correct, complete, consistent, and testable.
 7. **Requirement Management** — Control requirement changes, maintain traceability, and manage versions throughout the project lifecycle.
-

7. System & Design Engineering

7.1 What is Software Design?

Software Design is the **process of transforming requirements into a detailed blueprint of the system** that is to be built.

The design model describes:

- ✓ Software data structure
- ✓ Software architecture
- ✓ Interfaces
- ✓ Components

Design sits at the **intersection of:**

- Customer Requirements
- Business Rules
- Technical Considerations

Analysis modeling explains **WHAT** the system must do.

Design explains **HOW** the system will do it.

7.2. Elements of the Design Model

The **Design Model** consists of four major parts:

1. **Data Design**
2. **Architectural Design**
3. **Interface Design**
4. **Component (Module) Design**
5. **Deployment Level Design**

Together, they **form a complete design blueprint**.

1. Data Design

Definition

Data design transforms:

Analysis information (ERD, Data Dictionary, etc.) → implementation-ready data structures

Design always starts with data because:

- ✓ Data is the foundation for logic, components, and architecture.

Data design includes:

- Database structure
- Internal storage structures

- Data relationships
- Integrity & constraints

Some data design happens during architecture design. More detailed design happens during component design.

2. Architectural Design

Definition

Architectural design defines:

- ✓ Structure & relationships between major components
- ✓ Interaction patterns among subsystems
- ✓ High-level organization

Sources used:

- System Specifications
- Analysis Model
- DFDs & Use-Case Models

Architecture determines:

- System scalability
- Performance
- Reliability
- Maintainability



It also identifies **design patterns** that suit the problem context.

3. Interface Design

Interface design describes how:

- ✓ Software components communicate
- ✓ Users interact with the system
- ✓ System interacts with external systems

Sources used:

- Data flow diagrams
- Control flow diagrams
- Use-case interactions

Types of interfaces:

- User Interface (UI)

- System Interface (APIs)
 - Hardware Interfaces
 - Communication Interfaces
-

4. Component (Module) Level Design

Component design transforms:

Architectural blocks → Procedural logic & program structures

Inputs include:

- Process Specifications (PSPEC)
- Control Specifications (CSPEC)
- State Transition Diagrams (STD)

Component design defines:

- Algorithms
- Function definitions
- Local data structures
- Internal logic flow
- Error handling
- Dependencies



This is the **most detailed form of design**.

5. Deployment Level Design

Deployment design maps:

✓ Software subsystems → Physical hardware environment

It defines:

- Runtime environment
- Servers & nodes
- Network layout
- Load distribution

Failover strategy

7.3. Software Quality Attributes — FURPS

FURPS stands for:

F – Functionality

Quality of features, capability, & security.

U – Usability

Includes:

- User experience
- Ease of learning
- Documentation clarity
- Consistency
- Aesthetics

R – Reliability

Measured through:

- Failure rate
- Accuracy
- Mean Time To Failure (MTTF)
- Recoverability

P – Performance

Includes:

- Speed
- Response Time
- Resource consumption
- Throughput
- Efficiency

S – Supportability

Includes:

- Maintainability
- Extensibility
- Adaptability
- Serviceability

Supportability = Maintainability



7.1 Characteristics of Good Software Design

Good design is:

- Structured
 - Modular
 - Simple & readable
 - Maintainable
 - Extensible
 - Secure
 - Efficient
-

7.2 Core Design Concepts

Abstraction

Focus only on essential details.

Modularity

System is divided into manageable independent units.

Cohesion

Measures **how strongly tasks within a module relate.**

Higher cohesion = better quality.

Coupling

Measures **inter-dependency between modules.**

Lower coupling = better design.

Golden Rule

Good design = High Cohesion + Low Coupling

Layering

Separates:

- Presentation layer
- Business logic layer
- Data layer

Improving **maintainability, scalability, and security.**

8. Function-Oriented vs Object-Oriented Systems

7.1 Function-Oriented

- Focus on functions
 - Global data
 - Tight coupling
 - Top-down design
 - Poor scalability & reuse
-

8.2 Object-Oriented

Object = **data + behavior**

Core Principles:

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism

Benefits:

- Modularity
 - Reuse
 - Maintainability
 - Security
-



9. UML (Unified Modeling Language) & Design Models

9.1 What is UML?

UML is the **standard modeling language** used to visually represent the structure, behavior, and interactions of a software system.

It helps developers, analysts, and stakeholders **understand, design, and communicate system details clearly**.

9.2 Common UML Diagrams and Their Purpose

UML Diagram	Purpose
Use Case Diagram	Describes <i>how users (actors) interact with the system</i> and what functionality the system provides.
Class Diagram	Shows <i>classes, attributes, methods, and relationships</i> — representing the static object structure.
Sequence Diagram	Explains <i>the order of interactions</i> between objects over time.
Activity Diagram	Represents <i>workflows and process logic</i> using actions and flows.
State Diagram	Describes <i>how an object moves through different states</i> during its lifecycle.
Component Diagram	Shows <i>system modules and how they are connected or interact</i> .

9.3 Why UML is Important

UML helps in:

- Visual communication
- Reducing ambiguity
- Improving understanding among team members
- Supporting design validation before coding
- Providing documentation for future reference



10. OOAD — Object-Oriented Analysis & Design

OOAD is a **systematic approach to analyze requirements and design software using object-oriented concepts**.

10.1 Phases of OOAD

Phase 1 — Object-Oriented Analysis (OOA)

Focus: **What must the system do?**

Activities include:

- Identifying system actors (users or external systems)
- Identifying required functionalities using use cases
- Recognizing objects, their attributes, and roles
- Understanding business rules and constraints

Phase 2 — Object-Oriented Design (OOD)

Focus: **How will the system work internally?**

Activities include:

- Designing classes and objects
 - Defining attributes and methods
 - Establishing relationships (inheritance, association, aggregation, etc.)
 - Designing object interactions and workflows
-

10.2 Benefits of OOAD

- Improves **modularity and structure**
 - Enhances **code reusability**
 - Simplifies **maintenance and updates**
 - Aligns designs with **real-world entities**
 - Supports **scalable and flexible software development**
-

11. Coding Practices & Principles

Good coding practices ensure that software is:

- ✓ Maintainable
 - ✓ Readable
 - ✓ Scalable
 - ✓ Reliable
-

11.1 Core Programming Principles

SOLID Principles

- **SRP — Single Responsibility Principle**

A class should have **only one reason to change** (one clear responsibility).

- **OCP — Open/Closed Principle**

Software entities should be **open for extension but closed for modification**.

- **LSP — Liskov Substitution Principle**

A subclass must **behave correctly when used in place of its parent class**.

- **ISP — Interface Segregation Principle**

A client should **not be forced to depend on methods it does not use**.

- **DIP — Dependency Inversion Principle**

Depend on **abstractions**, not on concrete implementations.

Other Important Principles

- **DRY — Don't Repeat Yourself**

Avoid duplicate code. Put reusable logic in one place.

- **KISS — Keep It Simple**

Prefer simplicity over unnecessary complexity.

- **YAGNI — You Aren't Gonna Need It**

Do not implement anything until it is required.

11.2 Coding Conventions

Good coding conventions make programs **clean, readable, and maintainable**.

Key practices:

- Use **consistent and meaningful naming** for variables, methods, and classes
- Maintain a **clear and readable structure**
- Follow **proper indentation and formatting**
- Write **meaningful comments where needed**
- Avoid **deeply nested code blocks**
- Keep **functions small and focused** on a single task

Why They Matter

Clean coding:

- ✓ reduces defects
 - ✓ simplifies debugging
 - ✓ improves collaboration
 - ✓ lowers maintenance cost
 - ✓ increases system reliability
-

